REDIS

NOVEMBER 2026

King of Kings

TABLE OF CONTENTS

01 Introduction

02 Data Model

03 Query Language

04 Uses Cases + Strengths

05 Competitors

06 Drawbacks

07 Current System/Queries

08 Live Demo

Introduction



- Created by Salvatore Sanfilippo in early 2009
- Runs startup LLOOGG, which shows who's on their site
  - Every visitor = new row in MySQL, each row being reread over and over
- Rather than adding more servers, decided to try keeping it in memory
  - Does this by forking a copy of the program; the copy writes to disk, and the original keeps serving
- Allowed you to push structures/other data types into the cache
- February 2009, Redis is posted to an online forum: Hacker News.

DATA MODEL ASSUMPTIONS

- Key-Based Access
- No predefined schema
- Flexible Value Types
  - Eg. Strings, hashes, lists, sets, etc
- Relationships are application managed
- Denormalization is common
- Design data structures around queries

Query Languages

Supported Languages

Supports most large programming languages (C/C++, Python, Java, etc.)

Can support a SQL-like interface through the addition of libraries and extensions

Redis

Command-based, rather than declarative

Query Examples

MySQL:

```
SELECT *
FROM Furniture
WHERE ItemID = 101;
```

Redis:

```
HGETALL Furniture:101
```

USE CASES + STRENGTHS

- Because Redis stores the data in-memory, it is faster at retrieving and updating data.
- Best when you know exactly what you want it to do
- It also works well with temporary online data.
  - Data that quickly updates, or is ephemeral (live chats, leaderboards)
- Does well when used in conjecture with other database software

TikTok



- Used in live features:
  - Chat
  - Gifting
  - Content Moderation

Instagram



- Photo ID/User ID mapping for main feeds and activity feeds
- Supply data for user feeds

Online Shopping



- Track active users and their carts
- Manage temporary data:
  - Flash sale
  - Filtered serches

COMPETITORS

Top 3:

1. Valkey
2. DragonFly
3. KeyDB

	Redis	1. Valkey	2. DragonFly	3. KeyDB
Threading	Single-threaded	Multi-threaded	Multi-threaded	Multi-threaded
Licensing	Open source (with limits)	Open source	Open source (with limits)	Open Source
Redis Modules	✓	✓	✓	✓

Throughput Comparison:

SET	452,000 ops/sec	447,000 ops/sec	513,000 ops/sec	411,000 ops/sec
GET	460,000 ops/sec	524,000 ops/sec	462,000 ops/sec	634,000 ops/sec
Mixed	444,000 ops/sec	445,000 ops/sec	505,000 ops/sec	440,000 ops/sec

DRAWBACKS

Data Relationships

- Without additional support, Redis can't represent relationships between data points.
- Best you can do is have a key-value pair that references another pair/set.
  - Would have to update any writes in both places
- Other option is to include it all in the same hash

Long Term Records

- Redis is intended to be used for data that updates quickly and constantly.
- Active data is stored in RAM, so it would be expensive to hold on to large amounts of data

Current System

Redis Data

The data inserted into the database is saved into several csv files

Python/Docker

Docker locally hosts the server, and reads the Redis API directly, while Python uses its own librar

Python can be better for readability

Project Timeline

- Use either Docker or Python config file to set up the Redis server.
- Use Python to generate data for queries
- Then either run the queries through there respective files, or use corresponding Python files

First Query

Flash Sale!

Sets a sale price for a product for a set amount of time.

Attempts to order the product at the discounted price.

Second Query

Finds the average price of items sold in each shopping category over 60 days.

Third Query

Finds the average price of items sold in each shopping category over 60 days.

Queries

Q1:

```
SET stock:213 350
SET flashsale:213 199.99 EX 60
GET flashsale:213
TTL flashsale:213

EVAL "if redis.call('EXISTS', KEYS[1]) == 0 then return -1 end local left =
redis.call('DECRBY', KEYS[2], ARGV[1]) if left < 0 then redis.call('INCRBY', KEYS[2],
ARGV[1]) return -2 end redis.call('XADD', KEYS[3], '*', 'user_id', ARGV[2],
'product_id', ARGV[3], 'qty', ARGV[1]) return left" 3 flashsale:213 stock:213 orders 1
42 213

EXPIRE flashsale:213 0
GET flashsale:213

EVAL "if redis.call('EXISTS', KEYS[1]) == 0 then return -1 end local left =
redis.call('DECRBY', KEYS[2], ARGV[1]) if left < 0 then redis.call('INCRBY', KEYS[2],
ARGV[1]) return -2 end redis.call('XADD', KEYS[3], '*', 'user_id', ARGV[2],
'product_id', ARGV[3], 'qty', ARGV[1]) return left" 3 flashsale:213 stock:213 orders 1
42 213

GET stock:213
```

Q2:

```
TS.MRANGE 1789171200000 1794355199999 AGGREGATION avg 86400000 FILTER
kind=price GROUPBY category REDUCE avg
```

Q3:

```
FT.SEARCH id:products "@category:{electronics} @price:[50 300]" RETURN 3
name brand price SORTBY price ASC LIMIT 0 20
```

LIVE DEMO